

DAGSTER PIPELINES,
ENGINEERED

Dagster: Step-first resilience, minus the server.

Orchestration that scales with you.

+117k

downloads

**<50
MB**

RAM, total

SQLite

WAL state

THE CONTEXT

Where it hurts today

A service to run one flow

Type-checked but heavy

Slow onboarding curve

THE SHIFT

What you gain



Code-first

Pipelines are Python — versionable, testable.



Resume, don't restart

Every step checkpoints to the WAL.

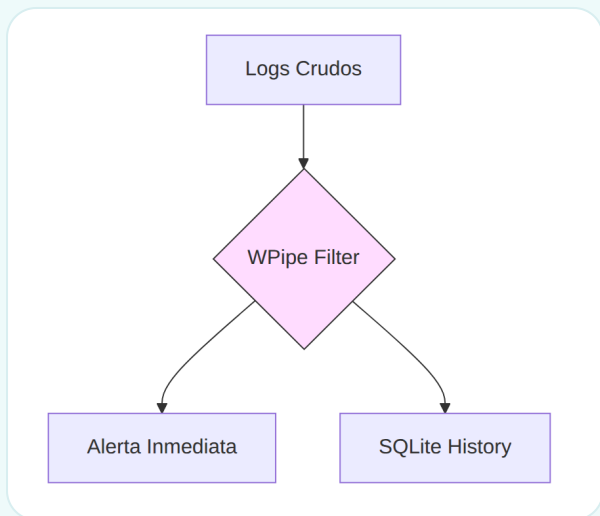


Featherweight

<50 MB. No broker, no daemon.

THE ARCHITECTURE

One pipeline, fully instrumented



State, retries and audit — inside a single run.

SIDE BY SIDE

wpipe vs. the status quo

Feature	WPipe	Dagster
Memory Footprint	< 50MB	> 500MB
Resilience	Local SQLite WAL	PostgreSQL / Cloud
Learning Curve	Low (Pythonic)	Medium/High
Auto-Docs	Built-in Mermaid	Dagster UI (Dagit)

SEE IT IN ACTION

Six lines. That's it.

```
from wpipe import Pipeline, step

@step(name="core",
version="v2.1")
def core(data):
    return {"v":
process(data["row"])}

```

CHECKPOINTS · RETRIES · TIMEOUT ·
PARALLEL · TRACKER

Try wpipe in your next pipeline ✓

Install the library, wrap your step
— checkpoints and docs appear
by themselves.

```
pip install wpipe
```